

ECE 251 Lab 8: Maskable Interrupts

Name: Zach Whitmore

Lab Day: Friday, 8 AM

Date: 12/08/2025

Objectives

- Learn what is an interrupt
- Learn the differences between maskable and nonmaskable interrupts
- How to use interrupts
- Implement interrupt service routines

Tools: VSCode, PlatformIO, Tiva board

Language: ARM assembly

List of Project files:

1. Lab8a.s
2. ECE215_asm_utils_new.s
3. setup.s

Procedure

Part 8.1: Completing the Digital Alarm Clock

I started from the provided Lab8.s file and focused on the basic structure of the program. I wrote an LED initialization routine for Port F based on the setup from Lab 7. This routine enables the clock for Port F, disables analog and alternate functions, sets PF1 through PF3 as outputs, enables digital I/O on those pins, and then writes 0 to the data register so all LEDs start in the off state.

Next I implemented the SysTick initialization subroutine `systick_ini` using the lab manual as a reference. I load a constant reload value of 3,999,999 into the SysTick reload register so that, with a 4 MHz SysTick clock, the timer generates an interrupt once every second. With the hardware timer and LEDs configured I moved on to the support routines for displaying and monitoring the clock. I wrote a display routine called `cdisp` that loads the address of the time string into R0, calls `OutStr`, and returns, so it simply prints the current time each time it is called.

For the alarm monitoring logic I implemented the `almon` routine. This routine loads the base addresses of the time string and the alarm string, then compares them character by character in a loop. On each iteration it reads a byte from each string and compares them. If the characters ever differ before reaching the end of the strings `almon` returns without doing anything. If it reaches the null terminator and all characters have matched that means the current time exactly matches the alarm time. In that case `almon` sets the `LedIsOn` flag in memory to 1. This flag is

used later in main to decide when to turn on the LEDs. After that I implemented the SysTick interrupt routine `sys_tick_handler`, which simply calls `clock`, `almon`, and `cdisp` in order and then returns.

Once the support routines were in place I finished the main program to tie everything together. In main I first call `InitUART` and print a greeting message to the terminal. I then call `_Init_PortF` to configure the LED alarm indicator on Port F. After that main prompts the user to set the clock and alarm by calling `setme` twice. Once with the time buffer and a “set clock” message, and once with the alarm buffer and a “set alarm” message. When both values have been entered I initialize `LedIsOn` to 0 so the alarm flag starts cleared, and then call `systick_ini` to start the SysTick interrupts and enable interrupts.

The last part of task 1 was to make the main stay in a loop that waits for the alarm and then for a key press to clear it. This is where I ran into one of my bigger bugs. At first I wrote a separate subroutine that executed the wait for interrupt (WFI) instruction and then, inside a polling loop in main, I manually called `sys_tick_handler`. This defeated the purpose of using interrupts, since the hardware is supposed to enter the ISR automatically when the timer expires, and it also risked interfering with the values being updated in the ISR. After going back through the logic I rewrote the wait loop so that main simply executes WFI, wakes up after each SysTick interrupt, checks the value stored in `LedIsOn`, and either goes back to sleep or handles the alarm.

While debugging that I also found a second bug. I was accidentally checking the address of `LedIsOn` instead of the value stored there. I loaded the pointer to `LedIsOn` into a register and compared that directly to 1, so the condition was never true and the program kept counting past the alarm time. Once I changed the code to load the byte at `LedIsOn` and compare that value, the program behaved as expected. When `LedIsOn` is detected as 1 main turns on the alarm indicator by writing 0x0E to the Port F data register, which lights all three LEDs. It then disables interrupts so the clock stops updating and calls `InChar_Echo` to wait for any key from the terminal. As soon as the user presses a key, main writes 0x00 to the Port F data register to turn off the LEDs and branches to the done routine.

Output:

```
Hello from Lab8!

Set the Clock Time (hh:mm:ss)
Enter hours (hh): 10
Enter minutes (mm): 10
Enter seconds (ss): 10
Enter A (for AM) or P (for PM): P
```

```
Set the Alarm Time (hh:mm:ss)
Enter hours (hh): 10
Enter minutes (mm): 10
Enter seconds (ss): 14
Enter A (for AM) or P (for PM): P
10:10:11 PM
10:10:12 PM
10:10:13 PM
10:10:14 PM
█
```

Figure 1. User Entry & Clock Stoppage at Alarm

Procedure

Part 8.2: Error Handling User Input

For part 2 I moved on to modifying the setme routine so it would only accept valid clock and alarm times. The original version of setme already prompted the user for hours, minutes, seconds, and either A or P, and stored those characters into the time or alarm string, but it did not check for bad input. At first I considered checking each field immediately after it was entered so if I typed an invalid hour like 00 the program would instantly print an error and reprompt just for the hours before moving on to minutes and seconds. While this would be very clear from the user's point of view, it made the setme routine much more cluttered because the error checking logic had to be mixed in between every InChar_Echo call.

To keep things cleaner I decided to let setme handle all of the prompting and input first, and then validate everything afterward in a separate subroutine. In my final version setme still prompts the user for hours, minutes, seconds, and A/P and builds the full "hh:mm:ss XM" string in memory. Once all four fields have been entered, setme calls a new subroutine called checkTime and passes it a pointer to the beginning of that string. The checkTime routine does all of the range checking in one place. It verifies that the hour characters are digits and that the numeric value is between 1 and 12, that minutes and seconds are digits between 00 and 59, and that the AM/PM character is either an "A" or a "P". If any of these checks fail checkTime returns a value indicating the time is invalid; if they all pass it returns a value indicating success.

Back in setme I looked at the return value from checkTime. If the time is invalid I print an "Invalid entry, try again." message and jump back to the top of the setme loop so the user has to re-enter hours, minutes, seconds, and AM/PM again. If the input is valid, setme simply returns to main and leaves the corrected time or alarm string in memory. This approach kept the

assembly code more organized and still gave clear feedback when I tested it by typing values like 00 for hours, out of range minutes or seconds, or the wrong letter for AM/PM. In all of those cases the program rejected the input, printed the error message, and reprompted until I entered a valid time.

Output:

```
Set the Clock Time (hh:mm:ss)
Enter hours (hh): 00
Enter minutes (mm): 10
Enter seconds (ss): 10
Enter A (for AM) or P (for PM): P
Invalid entry, try again.
```

Figure 2. Part 2: Error Handling Hours

```
Enter hours (hh): 10
Enter minutes (mm): 99
Enter seconds (ss): 10
Enter A (for AM) or P (for PM): p
Invalid entry, try again.
```

Figure 3. Part 2: Error Handling Minutes

```
Enter hours (hh): 10
Enter minutes (mm): 10
Enter seconds (ss): 99
Enter A (for AM) or P (for PM): P
Invalid entry, try again.
```

Figure 4. Part 2: Error Handling Minutes

```
Enter hours (hh): 10
Enter minutes (mm): 10
Enter seconds (ss): 10
Enter A (for AM) or P (for PM): k
Invalid entry, try again.
Enter hours (hh): █
```

Figure 5. Part 2: Error Handling AM/PM

Questions:

QUESTION 1: What are the differences between the initialization of the maskable and non-maskable interrupts?

Ans: Maskable interrupts are the ones the CPU is allowed to ignore. They have to be explicitly enabled by setting an interrupt enable flag before they can actually interrupt the CPU, so there's initialization involved. Nonmaskable interrupts are the ones the CPU never ignores, so they don't go through the same enable/disable process and are basically always active.

QUESTION 2: List the similarities and differences of interrupt service routines over the subroutines.

Ans:

Similarities:

- Both are blocks of code written like subroutines in assembly.
- Both usually end with BX LR.
- Both can call other subroutines.
- Both follow the same register usage rules.

Differences:

- A normal subroutine is entered with a BL instruction, an ISR is entered automatically by the hardware when an interrupt occurs.
- For a normal subroutine you decide what to push/pop, for an ISR the CPU automatically stacks R0 through R3, R12, LR, PC, and xPSR before running it.
- A normal subroutine just returns to the caller, an ISR returns from interrupt using a special LR value so the CPU restores the stacked registers and resumes the interrupted code exactly where it left off.
- An ISR is also responsible for clearing the interrupt flag so it doesn't immediately fire again, while a normal subroutine doesn't have that responsibility.

QUESTION 3: Which delay routine is more accurate, the routine that uses the software program or the one that uses the SysTick system? Explain why.

Ans: The SysTick delay is more accurate. A software delay is just a loop that goes through CPU cycles so the timing depends on the exact instructions, any code changes, and other interrupts that might run in the middle of it. SysTick uses a fixed hardware clock and a specific reload value so once I set it for a 1 second tick it keeps that timing regardless of what the main program is doing.

QUESTION 4: EXTRA CREDIT: Consider the program written in Figure 8.6. This program makes use of the SysTick system to generate a 10 second delay. The program tries to show the ‘START’ message at the start of the program. Then it waits for 10 seconds before displaying the ‘STOP’ message on the screen. The program will not work, as there are errors in the program. Locate the errors in the program and modify the program so that it will work properly.

Ans: I’ve uploaded a working ASM file of the working code. I fixed the following issues:

1. Changed RELOAD_VALUE to 3999999 to represent a 1 second delay to work with the defined delsec set at 10.
2. .syntax unified changed to .syntax unified
3. Added a branch link to InitUART at the start of main
4. Changed the SysTick control value from 0x01 to 0x03. With 0x01 only the SysTick timer itself was enabled so it was counting but never generating interrupts. With 0x03 both the timer and its interrupt are enabled, so the SysTick interrupt and the ISR can update the count variable.